



华中科技大学

操作系统原理课程设计报告

姓 名: Losyi
学 院: 计算机科学与技术学院
专 业: 计算机科学与技术
班 级: CS23
学 号: U2023xxxxx
指导教师:

分数	
教师	

2026 年 4 月 18 日

目 录

实验一 实现异常源代码行定位	1
1.1 实验目的	1
1.2 实验内容与实验记录	1
1.3 实验调试及心得	1
实验二 多核启动及运行	5
2.1 实验目的	5
2.2 实验内容与实验记录	5
2.3 实验调试及心得	5
实验三 堆空间管理	9
3.1 实验目的	9
3.2 实验内容与实验记录	9
3.3 实验调试及心得	9
实验四 多核内存并发管理实验指南	13
4.1 实验目的	13
4.2 实验内容与实验记录	13
4.3 实验调试及心得	13
实验五 信号量	18
5.1 实验目的	18
5.2 实验内容与实验记录	18
5.3 实验调试及心得	18
实验六 写时复制	20
6.1 实验目的	20
6.2 实验内容与实验记录	20
6.3 实验调试及心得	21
实验七 重载执行	24

7.1 实验目的	24
7.2 实验内容与实验记录	24
7.3 实验调试及心得	24

实验八 简易 Shell.....29

8.1 实验目的	29
8.2 实验内容与实验记录	29
8.3 实验调试及心得	29

实验一 实现异常源代码行定位

1.1 实验目的

本实验的具体要求为：通过修改 PKE 内核（包括 `machine` 文件夹下的代码），使得用户程序在发生异常时，内核能够输出触发异常的用户程序的源文件名和对应代码行。

在平时的内核开发过程中，当用户程序出现非法指令、内存访问越界这类异常时，内核通常只能捕获到异常发生时的程序计数器（PC）值，也就是一串枯燥的十六进制内存地址。所以这次实验，我们就是要通过解析编译器在编译阶段植入 ELF 文件中的 DWARF 调试信息，建立起二进制地址和源文件名和对应代码行之间的对应关系。

实现这个功能之后，一旦程序触发异常，内核就能直接打印出问题的源文件路径、具体行号，甚至是那一行的源代码。这不仅能大大提升操作系统的可调试性，也能让我们更深入地理解编译器、链接器和内核在处理元数据时是如何协同工作的，同时掌握工业级调试标准 DWARF 的解析方法。

1.2 实验内容与实验记录

本实验通过在内核中解析 ELF 文件的 DWARF 调试信息段，建立一套基于状态机机制的地址到源码映射表，并在异常处理流程中根据程序计数器实时定位并打印具体的源文件名、行号及对应代码内容。

1.3 实验调试及心得

1.3.1 实验原理分析与代码逻辑

实验原理的分析，解释新增/修改代码的作用，为什么要新增这些代码，自己的收获，对实验的建议等。

编译器在编译时会将源代码行号与机器指令地址的映射关系保存在 ELF 文件的 `.debug_line` 段中，采用 DWARF 标准格式。`.debug_line` 段采用状态机编码方

式，通过一系列操作码（`opcodes`）来高效地表示地址-行号映射关系。状态机维护当前的地址和行号，通过不同的操作码来更新状态并记录映射点。地址到源码的映射过程可概括如下：

编译时：源代码行号 → 编译器记录 → `.debug_line` 段

运行时：异常地址(PC) → 查询`.debug_line` → 源文件名+行号

因此，为达成实验目，我们需要先加载调试信息，在加载 ELF 文件时，读取`.debug_line` 段的内容，解析 DWARF 格式的调试信息，构建三个数据结构 `dir`、`file` 和 `line`。

具体而言，需要在 `process` 结构体中添加调试信息相关的字段。在 `elf.c` 中实现 DWARF 解析，加载调试信息，下面是实现 DWARF 状态机的部分代码片段

```
void make_addr_line(elf_ctx *ctx, char *debug_line, uint64
length) {
    process *p = ((elf_info *)ctx->info)->p;
    p->debugline = debug_line;

    // 分配三个数组的内存（在 debug_line 数据后面）
    p->dir = (char **)((((uint64)debug_line + length + 7) >
> 3) << 3);
    int dir_ind = 0, dir_base;

    // 添加默认目录"."在索引 0（DWARF 中 dir=0 表示当前目录）
    static char default_dir[] = ".";
    p->dir[dir_ind++] = default_dir;
```

由于 DWARF 使用 LEB128 的变长编码格式，我们还需要实现解码器：

```
// 无符号 LEB128 解码

void read_uleb128(uint64 *out, char **off) {
    uint64 value = 0;
    int shift = 0;
    uint8 b;
```

```

    for (;;) {
        b = *(uint8 *)(*off);
        (*off)++;
        value |= ((uint64)b & 0x7F) << shift;
        shift += 7;
        if ((b & 0x80) == 0) break; // 最高位为0 表示结束
    }
    if (out) *out = value;
}

```

异常时，在 `line[]` 数组中查找匹配的地址，通过文件索引找到源文件名和目录打开源文件，定位到对应行并读取内容。

1.3.2 实验调试与问题解决

在实验过程中，我遇到了以下核心问题并进行了针对性调试：

递归异常导致的死机：当 `print_error_line` 自身访问无效指针触发新异常时，会引发无限递归。我通过引入 `in_print_error_line` 标志位实现了重入保护，确保了内核的鲁棒性。

DWARF 版本不兼容：最初文件名显示为乱码，原因是编译器默认生成 DWARF 5 格式，其字符串存储在 `.debug_line_str` 段中。通过在 `Makefile` 中强制指定 `-gdwarf-4` 简化了解析难度，因为 DWARF 4 的字符串直接嵌入在段内。

行号计算偏移 5 行：通过逐字节对比 `readelf` 的原始数据，发现是由于结构体定义缺少 DWARF 4 的 `maximum_operations_per_instruction` 字段导致的数据错位。修复结构体定义后，行号解析恢复正常。

除此之外，还有文件索引越界，地址匹配失败，目录索引处理错误等问题，通过阅读标准文档，添加大量调试输出，解决了这些问题。

1.3.3 实验收获与建议

通过本次实验，我深入理解了编译器，调试器与内核之间的协作机制，掌握了 DWARF 调试规范的底层实现逻辑。

收获：学到了如何通过状态机解析二进制流，理解了变长编码（LEB128）

在资源受限环境下的优势。此外，通过阅读相关文档和 AI 辅助，我掌握了很多调试技巧，比如分阶段调试：先验证 DWARF 加载（打印 `line` 表内容）、再验证地址匹配（打印查找过程）、最后验证文件路径构建。并认识到异常处理的重要性，要考虑边界情况、NULL 情况、递归安全性等等。

实验二 多核启动及运行

2.1 实验目的

本实验的目标是打破 PKE 原有的单核运行限制，将其扩展为一个支持双核并发的操作系统环境。

在现代计算机体系结构中，多核处理器已成为主流。本实验希望让我们掌握多核操作系统的核心设计要点：资源共享与隔离、核心间同步以及分布式进程管理。通过修改内核，我们需要实现两个核心（Hart 0 和 Hart 1）的协同启动，为每个核心分配独立的运行资源，并支持它们分别加载并执行不同的用户程序。

2.2 实验内容与实验记录

本实验通过利用 RISC-V 的 `tp` 寄存器存储 `hartid` 实现核心识别，在 M 模式初始化中引入同步屏障确保共享资源不被重复初始化，并修改进程管理逻辑为每个核心提供独立的 `current` 指针与内存空间，最后利用原子操作实现多核协同退出机制。

2.3 实验调试及心得

2.3.1 实验原理分析与代码逻辑

在多核系统中，相同的代码镜像被加载到不同的物理核心上运行。为了让内核逻辑能够区分当前所在的硬件线程（以便分配私有资源），必须利用硬件提供的特性来持久化存储当前核心的唯一编号。RISC-V 体系结构为此专门预留了 `tp` 寄存器，作为核心私有的状态存储器。

因此，修改 `kernel/machine/minit.c` 代码：添加同步机制，只在 `hart0` 初始化共享资源。每个核心通过硬件读取自身的 `hartid`，并执行 `write_tp(hartid)` 将其存入 `tp` 寄存器。在后续的进程切换、中断处理（如 `strap.c`）中，只需通过 `read_tp()` 即可无开销地识别身份，这是实现后续所有隔离逻辑的基石

```
void m_start(uintptr_t hartid, uintptr_t dtb) {  
    // Only hart 0 initializes shared resources  
    if (hartid == 0) {spike_file_init(); init_dtb(dtb);}
```

```
// Synchronization barrier: wait for hart 0 to finish
sync_barrier(&init_sync_counter, NCPU);
sprintf("In m_start, hartid:%d\n", hartid);
// Store hartid in tp register
write_tp(hartid);} // ... rest of initialization
```

某些硬件资源在物理上是唯一的，例如模拟器的 HTIF 和设备树。如果两个核心同时尝试对其进行初始化，会导致寄存器状态混乱或总线竞争。因此，需要建立一个“同步点”，确保全局共享资源仅由主核心初始化一次，而其他核心在初始化完成前必须处于等待状态。在 `kernel/machine/minit.c` 中，我引入了基于原子操作的 `sync_barrier`。Hart 0 负责执行 `spike_file_init()` 等操作，而 Hart 1 通过自旋锁在屏障处阻塞，直到 Hart 0 完成任务并更新原子计数器。这确保了内核在正式运行前的硬件状态一致性。

```
static inline void sync_barrier(volatile int* counter, int
total) {
    __sync_fetch_and_add(counter, 1);
    while (*counter < total) {
        asm volatile("nop");}}
```

单核环境下，全局变量 `current` 用于追踪运行进程，但在双核下这会引发数据竞争。解决思路是将这些关键状态变更为数组形式。同时，为了防止两个核心在陷入内核时互相践踏对方的执行流，必须通过逻辑地址偏移，为每个核心划分专属的内核栈、用户栈和加载空间。

因此需要实现 PCB 扩展和内存规划：

将 `current` 声明为 `struct process* current[NCPU]`，通过 `current[read_tp()]` 访问当前核进程。

在 `kernel.c` 中，利用 `hartid` 计算地址偏移。例如，Hart 0 的 `Trapframe` 位于 `0x81300000`，而 Hart 1 偏移 4KB 位于 `0x81301000`；用户程序加载地址更是通过 `hartid * 64MB` 的巨量偏移（`0x81000000` 与 `0x85000000`）实现完全解耦。

表 2-1 内存布局设计表

资源	HART0 地址	HART1 地址	偏移量
Trapframe	0x81300000	0x81301000	+0x1000 (4KB)

资源	HART0 地址	HART1 地址	偏移量
内核栈	0x81200000	0x81210000	+0x10000 (64KB)
用户栈	0x81100000	0x81110000	+0x10000 (64KB)
程序加载	0x81000000	0x85000000	+0x4000000 (64MB)

最后我们要实现多核协同退出机制，多核环境下的关机操作极具危险性——如果一个核心执行完任务立刻调用 `shutdown`，会强行杀掉另一个仍在运行的核心。我们可以采用集合机制：每个核心在退出时进行登记，只有当所有核心都登记完毕，才允许主核心执行最终的关机指令。据此，在 `sys_user_exit` 函数中，引入全局原子计数器 `exit_counter`，最终由 Hart 0 检查到 `exit_counter == NCPU` 后打印关机信息并退出。

```

ssize_t sys_user_exit(uint64 code) {
    uint64 hartid = read_tp();
    sprintf("hartid = %d: User exit with code:%d.\n", hartid,
code);
    // Atomically increment exit counter
    __sync_fetch_and_add(&exit_counter, 1);
    // Wait for all harts to exit
    while (exit_counter < NCPU) {asm volatile("nop");}
    // Only hart 0 performs shutdown
    if (hartid == 0) {
        sprintf("hartid = %d: shutdown with code:%d.\n", hartid,
code);shutdown(code);}
    // Other harts wait indefinitely
    while (1) { asm volatile("wfi");}}

```

2.3.2 实验调试与问题解决

在实验过程中，我遇到了以下核心问题并进行了针对性调试：

退出时系统死锁无法打印 shutdown: 程序在两个核心都打印 User exit 后便不再有后续动作，初期因使用 `exit_counter++` 非原子操作，导致两个核心并发修改变量时产生读写竞争，计数值仅为 1 未达到 `NCPU=2` 的阈值，进而让所

有核心在 while 循环中永久自旋，我通过改用__sync_fetch_and_add 强制硬件执行原子加法，确保计数值能准确反映退出核心的总数，解决了死锁问题。

HartID 混乱导致 Hart 1 显示 ID 为 0: 用户态程序运行后 Hart 1 打印的 hartid 前缀错误，经查是 do_fork 或 load_user_program 时未将 tp 寄存器的值保存至 trapframe 中，内核通过 sret 返回用户态时 tp 寄存器丢失了原本的 ID 信息，我在加载应用和上下文切换逻辑中明确设置 proc->trapframe->tp = hartid，保证了核心身份信息在内核态与用户态之间同步传递，修复了 ID 混乱问题。

内存访问异常 (Load/Store Access Fault) 导致 Panic: 原因是最初设置的内核栈 (kstack) 和用户栈 (user_stack) 偏移量仅 4KB 过小，多核并发执行时 Hart 1 的栈向下增长踩到了 Hart 0 的数据区域，我将内核栈和用户栈的偏移量统一扩大到 64KB (0x10000)，为多核运行提供了充足的缓冲隔离空间，解决了内存异常问题。

2.3.3 实验收获与建议

通过本实验，我理解了多核操作系统的基本原理，包括资源的共享与隔离，核心间的同步与协作、并发执行的管理。还了解了 hartid 的概念和 tp 寄存器的作用，原子操作和同步机制。这些技术是现代多核操作系统的基础，为后续的进程调度、负载均衡、核间通信等高级功能打下了坚实基础。

实验三 堆空间管理

3.1 实验目的

本实验目标是实现一套高效且紧凑的堆空间管理机制。

在 lab2_2 中,内存分配采用的是极度简化的“页级分配”策略,即每次 `malloc` 请求都会直接分配并映射一个完整的 4KB 物理页,这导致了严重的内存内部碎片。我们需要打破这一局限,通过在内核中引入内存控制块指标体系,实现细粒度的动态内存管理,使内核具备类似工业级 `malloc` 的堆管理能力,提升系统的内存利用率。

3.2 实验内容与实验记录

本实验通过在进程控制块中扩展堆管理元数据字段,并设计一套基于单向链表的内存控制块(MCB)管理算法,实现支持空闲块首适配复用、跨页按需补齐映射以及 8 字节对齐的安全内存分配与回收系统。

3.3 实验调试及心得

3.3.1 实验原理分析与代码逻辑

要实现紧凑堆管理,同时考虑 PKE 的简化特性:没有复杂的 `sbrk/brk`、`mmap` 子系统,但我们可以在系统调用里做“最小足够”的堆管理;用户态通过 `better_malloc(n) / better_free(p)` 触发系统调用;系统调用在内核态可访问进程页表并完成映射。用户程序需要复用释放的块,需要紧凑布局(两个分配间距不超过 512)

基于这些约束,我们可以在堆区域中引入一个内存控制块 MCB (作为每个块的元数据。MCB 作为元数据,记录了该块的大小、占用状态及指向下一块的指针。通过将这些 MCB 串联成链表,内核可以像遍历账本一样管理整个堆区域。因此,我们在 `kernel/process.h` 的 `process_t` 结构中新增了 `heap_start` (堆起点)、`heap_current` (当前堆顶虚拟地址)和 `mcb_head` (链表首节点物理指针)。在首次调用 `better_malloc` 时,内核会映射首个物理页,并在页首初始化一个头 MCB。任何返回给用户的指针实际上都是 `mcb_va + sizeof(MCB)`

```
typedef struct process_t {
```

```

uint64 kstack;

pagetable_t pagetable;

trapframe* trapframe;

// 堆管理相关字段 - added for Lab2_challenge2

uint64 heap_start;    // 堆起始虚拟地址

uint64 heap_current;  // 当前堆顶（已分配的最高地址）

mcb* mcb_head;        // MCB 链表头指针

} process;

```

接着考虑内存分配策略，为了实现高效的空间利用，分配算法应优先考虑废物利用。当用户请求内存时，内核首先执行首次适配扫描，遍历 MCB 链表寻找状态为空闲且大小满足要求的块。只有当链表中所有空闲块都不满足条件时，内核才会从当前的堆顶向高地址方向追加一个全新的块。

下面代码展示了先扫描空闲块再追加新块的核心逻辑，在 `sys_user_better_malloc` 系统调用中，内核遍历由 `mcb_head` 起始的链表。若发现 `m_stat == 0` 且 `m_size >= size`，则直接标记 `m_stat = 1` 并返回。若需追加新块，则在 `heap_current` 处写入新 MCB，并根据请求大小（需 8 字节对齐）向后推进 `heap_current` 指针。

```

kernel > C syscalls > sys_user_better_malloc(uint64)
64  uint64 sys_user_better_malloc(uint64 size) {
98
99      // 遍历 MCB 链表，寻找空闲块
100     mcb* prev = NULL;
101     mcb* curr = current->mcb_head;
102
103     while (curr != NULL) {
104         if (curr->m_stat == 0 && curr->m_size >= size) {
105             // 找到合适的空闲块
106             curr->m_stat = 1; // 标记为已分配
107
108             // 计算返回给用户的虚拟地址 (MCB 后面的内存)
109             // 计算 MCB 的虚拟地址
110             uint64 mcb_va = 0;
111             uint64 search_va = current->heap_start;
112             mcb* search_mcb = current->mcb_head;
113
114             // 遍历找到 curr 对应的虚拟地址
115             while (search_mcb != NULL && search_mcb != curr) {
116                 uint64 block_size = sizeof(mcb) + search_mcb->m_size;
117                 search_va += block_size;
118                 void* next_pa = user_va_to_pa((pagetable_t)current->pagetable, (void*)search_va);
119                 search_mcb = (mcb*)next_pa;
120             }
121             mcb_va = search_va;
122
123             return mcb_va + sizeof(mcb);
124         }
125         prev = curr;
126         curr = curr->next;
127     }
128
129     // 没有找到空闲块，需要分配新块
130     uint64 total_needed = sizeof(mcb) + size;
131     uint64 new_mcb_va = current->heap_current;
132

```

图 3-1 sys_user_better_malloc 关键代码

跨页分配的目标是虚拟地址仍然从 `heap_current` 连续增长，需要映射的页按需映射，新块 MCB 仍然写在 `heap_current` 位置，而不是跳到新页开头。当追加新块导致所需的结束虚拟地址 (`end_va`) 跨越 4KB 边界时，内核不能简单地将块挪到新页开头，否则会造成巨大的地址跳跃。正确的思路是保持虚拟地址的绝对连续，即 MCB 紧跟在前一个块之后，内核仅负责检测并补齐该地址段所覆盖的所有缺失物理页映射。

内核通过计算 `end_va = new_mcb_va + sizeof(MCB) + size` 确定覆盖范围，并计算对应的 `end_page`。随后通过循环遍历从 `new_mcb_va` 到 `end_va` 涉及的所有虚拟页，利用 `page_walk` 检查 PTE 是否有效。若未映射，则调用 `alloc_page` 配合 `user_vm_map` 逐页建立映射，确保硬件 MMU 能够正确解析这段连续的虚拟空间。

内存释放和对齐比较简单，这里不再赘述。

3.3.2 实验调试与问题解决

在实验过程中，我遇到了以下核心问题并进行了针对性调试：

跨页分配致地址跳跃：针对跨页分配导致的虚拟地址跳跃问题，我最初错误地将跨页块的 MCB 直接放置在新建物理页的起始处，这导致两个连续分配的内存块之间产生了接近 4KB 的地址鸿沟，违反了紧凑性检查。通过分析发现，这是由于过度关注物理页边界而忽略了虚拟地址空间的抽象性，通过修改逻辑使新块永远从 `heap_current` 连续追加，并仅在 `end_va` 跨页时通过循环补齐映射，最终实现了虚拟地址的精确管理，使块间距成功缩减至 MCB 头部开销加对齐字节。

misaligned AMO 异常：关于频繁出现的 misaligned AMO 硬件异常，其根源在于我最初未对用户请求的 `size` 进行 8 字节对齐处理，导致 `heap_current` 推进到了非对齐边界，使得后续写入包含 `uint64` 成员的 MCB 结构体时发生了非法内存访问。通过在内核入口对 `size` 执行按 8 取整操作，并确保 `sizeof(MCB)` 也是对齐的，我保证了所有堆元数据的写入都落在合法的内存地址上，从而彻底解决了这一稳定性问题。

存在映射冲突隐患：在实现过程中曾出现由于忘记推进全局虚拟页指针 `g_ufree_page` 而导致的映射冲突隐患。如果内核在为堆映射新物理页后不及时更新 `g_ufree_page`，后续其他进程或本进程的其他映射请求可能会得到相同的虚拟基址，引发覆盖现有页表的严重后果，通过在每次 `user_vm_map` 成功后立即按

页大小推进该指针，确保了系统级虚拟页分配的单调递增和唯一性。

地址混淆致用户无法访问：在内核态管理用户堆时，由于混淆了虚拟地址（VA）和物理地址（PA），一度导致返回给用户的指针在用户态下无法访问。这是因为内核在遍历链表时需要使用内核可访问的物理地址（由 `user_va_to_pa` 获取），但最终必须将计算出的虚拟地址返回给系统调用，通过在代码中严格区分变量命名的地址域，我确保了“写元数据”使用 PA、“返回用户”使用 VA，解决了跨地址空间的读写混乱。

3.3.3 实验收获与建议

本次实验让我对堆这一抽象概念有了深刻的认识：堆并非天然存在，而是内核通过管理元数据在原始虚拟空间中维护出来的。我掌握了如何通过 MCB 链表实现空闲块复用、如何利用页表遍历实现按需映射、以及在 64 位 RISC-V 架构下遵循地址对齐约束的重要性。

实验四 多核内存并发管理实验指南

4.1 实验目的

本实验需要在一个支持双核的 RISC-V 操作系统内核中，实现并发安全的物理内存分配与回收，并支持每个核心独立加载运行不同的用户程序。在多核环境下，物理内存作为全局共享资源，其分配与回收过程极易产生数据竞争，如多个核心同时修改空闲页链表可能导致内存泄漏或系统崩溃。

因此，本实验的目标是通过硬件级的原子指令实现互斥锁，保护物理内存分配器；同时，通过扩展进程控制块（PCB）和改进 ELF 加载逻辑，实现不同核心独立加载运行不同的应用程序，并确保各进程的堆空间在虚拟地址空间中完全隔离，最终建立一套支持多核协同运行与安全退出的内存并发管理体系

4.2 实验内容与实验记录

本实验通过引入基于 amoswap 原子指令的自旋锁保护物理页分配，并利用 tp 寄存器标识核心身份以实现应用程序的差异化加载与堆空间私有化管理。

4.3 实验调试及心得

4.3.1 实验原理分析与代码逻辑

在单核操作系统中，物理内存分配器只需要考虑串行访问。但在多核环境下，多个处理器核心并发执行，它们共享物理内存资源。如果不加控制地同时访问共享资源（如物理页分配器），会导致严重的数据竞争。因此，每个核心运行的用户进程应当拥有独立的虚拟地址空间（通过页表实现），独立的堆空间指针（避免虚拟地址冲突）和独立的程序代码和数据（加载不同的 ELF 文件）。

此外还有同步机制需求，需要依靠互斥锁，原子操作和内存屏障实现。

首先，我们需要实现物理内存管理的并发安全，多核架构中，alloc_page 和 free_page 会并发操作全局空闲链表 g_free_mem_list，这属于典型的临界区访问。若无保护，可能导致两个核心获得同一个物理页。为此，需要引入互斥锁（Mutex），利用硬件提供的原子性保证同一时刻只有一个核心能执行链表操作。

在 kernel/pmm.c 中，我使用 RISC-V 的 amoswap.w.aq 指令实现自旋锁的获取），并用 amoswap.w.rl 实现释放。通过将 pmm_lock 声明为 volatile，确

保了多核环境下变量可见性，有效地防止了物理页的重复分配。

```
// 获取锁（自旋等待）

static void pmm_lock_acquire() {

    int old = 1; // 预设为 1，用于检测锁的状态
    // 自旋循环：不断尝试获取锁

    while (1) {

        asm volatile("amoswap.w.aq %0, %1, (%2)"

                     : "=r"(old) // 输出：将交换前的旧值入 old
                     : "r"(1), // 输入：将 1 写入内存
                       "r"(&pmm_lock) // 输入：目标内存地址
                     : "memory"); // 副作用：修改内存

        // 如果旧值是 0，说明之前锁是空闲的，我们成功获取了锁

        if (old == 0) break;

        // 否则，锁被占用，继续自旋等待

    }

}
```

接着，需要实现多核应用程序独立加载。Spike 模拟器通过命令行参数传递多个应用路径（argv 数组）。单核逻辑通常硬编码加载 argv，但在双核环境下，必须根据当前运行核心的 hartid 选择对应的索引，从而让 Hart 0 运行 app0，Hart 1 运行 app1。

修改 kernel/elf.c 中的 load_bincode_from_host_elf 函数，通过 read_tp() 系统底层接口读取当前核心存储在 tp 寄存器中的 ID，并将其作为 argv 的下标。这种基于身份标识的动态映射，解决了多核运行同一程序的逻辑冲突。以下是部分代码片段。

```
void load_bincode_from_host_elf(process *p) {

    arg_buf arg_bug_msg; // 解析命令行参数

    size_t argc = parse_args(&arg_bug_msg);

    if (!argc)

        panic("You need to specify the application program!\n");

    // 【关键修改 1】：使用 read_tp() 获取当前核心 ID

    uint64 hartid = read_tp();

}
```

```

    // 【关键修改 2】：根据 hartid 打印对应的应用程序名称
    sprintf("hartid = %ld: Application: %s\n", hartid, arg_bug_
_msg.argv[hartid]);

    // elf loading. elf_ctx is defined in kernel/elf.h
    elf_ctx elfloader;
elf_info info;

    // 【关键修改 3】：根据 hartid 加载对应的应用程序文件
    // Hart 0 加载 argv[0], Hart 1 加载 argv[1]
    info.f = spike_file_open(arg_bug_msg.argv[hartid], O_RDONLY, 0); info.p = p;
}

```

若使用全局数组管理堆指针，会破坏进程的封装性并导致多进程环境下的逻辑混乱。为了支持进程隔离，每个进程应当拥有独立的堆空间状态，避免虚拟地址在不同核心间发生重叠或冲突。我将 `user_heap_ptr` 从全局变量移入 `process_t` 结构体中，作为进程的私有属性。在 `sys_user_allocate_page` 中，通过 `current[hartid]->user_heap_ptr` 访问当前进程的堆顶，确保了每个进程在物理内存共享的前提下，虚拟地址空间保持绝对隔离。

```

typedef struct process_t {
    uint64 kstack;           // 内核栈
    pagetable_t pagetable;   // 页表
    trapframe *trapframe;    // 陷入帧
    uint64 user_heap_ptr;     // 【新增】用户堆指针
} process;

```

最后是多核协同退出与同步屏障，这与实验二时的实现思路基本一致，这里不再赘述。

需要注意的是，为了配合 `user/app_alloc0.c` 和 `user/app_alloc1.c` 的测试逻辑，我们将 `naive_malloc` 修改为请求一个固定较大的块（4000 字节），这样每次 `malloc` 基本都会触发一次新的页面分配。

```

void* naive_malloc() {
    // 参数说明：

```

```

// SYS_user_allocate_page: 系统调用号
// 4000: 请求的内存大小 (字节)
//      接近一个页面大小 (4096), 确保分配新页
// 其他参数: 未使用, 填 0
return (void*)do_user_call(SYS_user_allocate_page, 4000,
0, 0, 0, 0, 0, 0);
}

```

4.3.2 实验调试与问题解决

在实验过程中，我遇到了以下核心问题并进行了针对性调试：

双核加载运行同一程序：在初始调试中发现 Hart 1 也在运行 `app_alloc0`，导致输出中只有 `==` 前缀而无 `>>>` 前缀；通过排查 `kernel/elf.c` 发现加载逻辑硬编码使用了 `argv`，于是引入 `read_tp()` 获取 `hartid` 并将其作为数组索引，成功实现了不同核心加载对应应用的正确逻辑。

重构导致的编译报错：在将全局变量 `g_ufree_page` 重构为进程私有的 `user_heap_ptr` 后，编译提示 `process has no member named ufree_page`；通过全局搜索发现 `kernel/vmm.c` 中仍引用了旧的字段名，通过同步修改所有引用位置，解决了变量名不一致导致的重构遗留问题。

物理内存重复分配隐患：虽然应用规模较小时竞争窗口窄，但理论上不加锁会导致数据破坏；我在 `pmm.c` 中利用内联汇编实现了包含 `.aq` 和 `.rl` 后缀的 `amoswap` 指令锁，并通过在临界区人为加入延迟进行压力测试，验证了锁机制在极端并发情况下防止物理页重复分配的有效性。

头歌评测机比对格式差异：功能实现正确但头歌评测报错，经 `diff` 对比发现输出字符串如 `code: 0` 缺少空格，且部分日志缺少 `hartid` 前缀；我根据预期输出精确调整了 `sprint` 中的格式字符串，确保了空格、冒号等细节与头歌标准完全匹配，最终通过了逐字符对齐的严格比对

4.3.3 实验收获与建议

通过本次实验，我掌握了很多并发控制和多核编程的知识。掌握了自旋锁的实现原理，学会使用 RISC-V 原子指令 `amoswap`，理解了内存屏障的作用。此外，还加深了对操作系统课学的进程管理相关知识的认识，学会自己设计 PCB，

实现资源管理，地址隔离和内存分配。

实验五 信号量

5.1 实验目的

本实验要求在 PKE 内核中实现信号量机制，使用户态程序能够通过系统调用完成基本的进程同步。实验完成后，用户程序应支持创建信号量，并执行 P/V 操作，从而实现多个进程之间按指定顺序运行。本实验不仅要求我们掌握 P/V 操作的底层实现逻辑，更要求我们将信号量的阻塞唤醒机制深度融入现有的进程调度流中，从而实现进程间高效、有序的协作。

5.2 实验内容与实验记录

本实验通过在内核中定义静态信号量池及 FIFO 等待队列，实现了 sem_P 阻塞调度与 sem_V 唤醒入队的核心逻辑，并通过系统调用分发机制使应用进程能够完成顺序同步与生产者-消费者模型的复杂协作。

5.3 实验调试及心得

5.3.1 实验原理分析与代码逻辑

由于本实验建立在已有的进程管理与调度框架上，因此重点不在于重新设计调度器，而在于把信号量的“阻塞—唤醒—重新入队”过程准确接入现有内核执行流。

信号量是一种经典的同步原语，本质上可以看成“计数器 + 等待队列”的组合。为了实现这一机制，内核需要记录当前可用的资源数（count），并维护一个记录哪些进程正在等待该资源的队列。为了简化内存管理并提高效率，实验采用静态数组维护一个信号量池。此外，进程结构体（process_t）需要扩展一个指针成员，用于在不引入额外节点的情况下，将阻塞进程直接链接成单向 FIFO 队列。

具体实现上，在 kernel/process.h 中定义 semaphore_t 结构体，包含 id、count 以及队列头尾指针 queue_head/tail。同时在 process_t 中新增 sem_queue_next 指针。通过这种设计，sem_new 可以在信号量池中遍历 id == -1 的空闲槽位并初始化计数器，为用户程序提供一个合法的标识符。

```
typedef struct semaphore_t {
```

```

    int id;
    int count;

    process *queue_head;
    process *queue_tail;
} semaphore_t;

```

接着我们需要实现 PV 操作，在 P 操作中完成阻塞和调度切换，在 V 操作中完成唤醒和重新入就绪队列。

P 操作实现了按需申请。当进程尝试获取资源时，如果 count 减一后小于 0，说明资源已被耗尽，此时进程不能继续空转，而必须进入阻塞状态。为了保证公平性，被阻塞的进程应挂入对应信号量的队尾，随后内核必须立即调用调度器，将执行权移交给其他就绪进程。对应代码骨架为：

```

semaphores[sem_id].count--;
if (semaphores[sem_id].count < 0) {
    current->status = BLOCKED;

    ...

    schedule();
}

```

V 操作实现了资源归还与通知。当进程释放资源使 count 加一后，如果 count <= 0，说明在此之前已有进程在队列中排队等待。此时，内核必须按照 FIFO 原则从队列头部取出一个进程，将其状态恢复为就绪，并将其重新放入系统的就绪队列中，使其在未来的调度中能够再次获得执行机会。

最后，别忘了在内核中注册对应的系统调用号，并在 do_syscall 中建立分发映射。同时，用户态库函数应屏蔽底层 ecall 指令的细节，提供符合 C 语言习惯的函数接口。

5.3.2 实验调试与问题解决

本实验整体实现较直接，调试过程也相对简短，主要问题如下：

process_t 类型定义冲突： 在定义 semaphore_t 结构体时，如果直接引用 process * 成员，会因为 process.h 中的循环依赖或定义顺序问题导致编译器报错。我在 kernel/process.h 中通过添加前向声明 typedef struct process_t process; 明确了类型信息，从而消除了结构体相互引用时的未定义错误。

信号量池溢出导致内核崩溃：实验初期为了保险将信号量池上限设得过大，结果导致内核 BSS 段占用空间剧增，超出了预设的 `kernel_size` 限制，导致加载失败。通过参考文档建议，我将信号量数量上限调整到一个合理的范围（如 16 或 32），既满足了测试应用对多个信号量的需求，又保证了内核镜像体积的紧凑性。

生产者-消费者死锁风险：在实现 `sem_P` 时，如果未能在将进程挂入等待队列后立即执行 `schedule()`，或者在 `sem_V` 中唤醒进程后忘记将其插入就绪队列，会导致所有进程都进入阻塞或无法被调度的状态。通过在代码关键点增加 `sprint` 打印进程状态切换日志，我发现 `sem_V` 唤醒逻辑中漏掉了 `insert_to_ready_queue` 调用，修正后成功实现了父子进程间的资源生产与消费同步

5.3.3 实验收获与建议

本次实验让我对操作系统进程同步机制有了更为具体的理解。通过亲手编码实现 P/V 操作，我清晰掌握了进程阻塞与唤醒的完整流程，准确理解了信号量计数器的数值含义，同时将同步原语与进程调度知识有机结合，不再停留在抽象概念层面。

实验六 写时复制

6.1 实验目的

本实验的目标是为 PKE 内核中的 `fork` 机制加入写时复制支持。基础版 `fork` 在复制堆空间时，会直接为子进程分配新物理页，并把父进程的数据整页拷贝过去。这种实现虽然正确，但效率不高。因为在很多场景中，子进程 `fork` 之后并不会立刻修改堆数据。如果一开始就把所有堆页完整复制，会造成明显的内存浪费和不必要的搬运开销。

因此，本实验的核心思想是：在 `fork` 发生时不立即复制物理页，而是让父子进程先共享同一物理页。只有当某个进程第一次对这页进行写操作时，才在页故障处理中真正复制页面。

6.2 实验内容与实验记录

本实验通过在页表项中使用软件定义的 `PTE_COW` 标志位及建立物理页引用计数体系，将 `do_fork` 中的堆段复制逻辑改造为只读共享映射，并配合 `CAUSE_STORE_PAGE_FAULT` 异常处理实现按需分配新页与数据分离。

6.3 实验调试及心得

6.3.1 实验原理分析与代码逻辑

写时复制可以概括为两句话：

第一，fork 时先共享。

第二，写入时再分离。

换成页表语言描述就是：原本父进程堆区中的可写页，在 fork 后不再保持可写，而是变成“可读不可写，带 COW 标记”的共享页。父进程和子进程的页表项都指向同一个物理页。由于该页已经失去写权限，所以无论父进程还是子进程，只要第一次尝试写它，就会触发 Store Page Fault。内核在处理这个页故障时，检查该页是否为 COW 页。如果是，则分配新页、复制旧内容、更新当前进程页表，然后恢复写权限。这样，页复制动作就被延迟到了“真正发生写入”的那一刻。

为了在写入时捕获请求，内核在 fork 阶段必须剥夺父子进程对共享页的写权限（去掉 PTE_W），使其变为只读。但仅仅只读不足以区分普通非法写入和合法的 COW 请求，因此需要利用 RISC-V 页表项中保留的软件位（RSW）来标记该页属于写时复制状态。我们可以在 kernel/riscv.h 中定义 PTE_COW 为第 8 位。在 vmm.c 的 map_cow_page 中，映射时不加入 PTE_W，而是打上 PTE_COW 标记。这确保了任何写操作都会触发 Store Page Fault，从而将控制权交给内核的修复逻辑。

当多个进程共享同一物理页时，内核必须知道该页被引用的次数。当某个进程发生 COW 导致分离时，旧物理页的引用计数应减一；当计数降为 1 时，说明该页已由某个进程独占，此时若再发生写故障，则无需复制，直接恢复写权限即可。此外，引用计数还能防止父进程退出时错误地释放仍被子进程使用的物理页。通过在 kernel/pmm.c 中增加 page_ref_count 数组，并实现 page_ref_inc、page_ref_dec 和 page_ref_get 接口。每当在 do_fork 中建立一次 COW 映射，就增加对应物理页的计数；在故障修复或进程销毁时递减计数，例如 page_ref_dec 代码为：

```
void page_ref_dec(void* pa) {
    int idx = pa_to_index(pa);
    if (idx >= 0 && idx < MAX_PAGES && page_ref_count[idx] >
0) {
```

```

    page_ref_count[idx]--;
    if (page_ref_count[idx] == 0) {
        free_page(pa);}}}

```

本实验的最后一个重点是，我们要实现写故障修复与页表重建逻辑，当 CAUSE_STORE_PAGE_FAULT 发生时，内核需要完成申请新页、数据拷贝、更新页表、刷新 TLB 的完整链路。关键在于，新映射的页表项必须恢复 PTE_W 权限，并且为了兼容硬件特性，还必须补全访问位和脏位，否则可能导致连续触发故障。在 kernel/vmm.c 中实现 handle_cow_fault 函数，这是实验中最核心的处理函数，该函数首先通过 page_ref_get 判断是否需要物理复制：若需复制，则通过 alloc_page 申请新物理页并使用 memcpy 搬运数据；若不需复制，则直接修改原 PTE 权限。最后必须调用 local_flush_tlb_all() 刷新 TLB，确保 CPU 立即看到更新后的映射关系。

```

int handle_cow_fault(pagetable_t pagetable, uint64 va) {
    va = va - (va % PGSIZE);
    pte_t *pte = page_walk(pagetable, va, 0);
    if (pte == 0 || (*pte & PTE_V) == 0) return -1;
    if (!(*pte & PTE_COW)) return -1;
    uint64 pa = PTE2PA(*pte);
    int ref_count = page_ref_get((void*)pa);
    if (ref_count > 1) {void* new_pa = alloc_page();
        if (new_pa == 0) return -1;
        memcpy(new_pa, (void*)pa, PGSIZE);
        *pte = PA2PTE((uint64)new_pa) | PTE_V | PTE_R | PTE_W |
PTE_U | PTE_A | PTE_D;
        page_ref_dec((void*)pa);
        page_ref_inc(new_pa);
    } else {
        *pte = PA2PTE(pa) | PTE_V | PTE_R | PTE_W | PTE_U | PTE
_A | PTE_D;
    }
    flush_tlb();
    return 0;}

```

6.3.2 实验调试与问题解决

在实验过程中，我遇到了以下核心问题并进行了针对性调试：

引用计数数组越界报错：在初期运行 `app_cow` 时，内核打印出的引用计数始终为 0，导致无法正确触发物理页复制逻辑。通过分析 `page_ref_inc` 的调试信息发现，物理页索引（`index`）超过了预设的 `MAX_PAGES` 范围，这是由于 `pmm.c` 中对物理内存总量的静态评估不足导致的。解决方法是将 `MAX_PAGES` 从 8192 扩大到能覆盖全部物理内存的范围，确保每一个物理页的引用状态都能被准确记录在数组中。

PTE 位缺失导致重复故障：在实现 COW 修复逻辑后，程序虽然能分配新物理页并返回用户态，但同一地址会立即再次触发 `Store Page Fault`，最终导致程序崩溃。经过对比硬件规范发现，新生成的页表项虽然开启了 `PTE_W`，但忘记设置 `PTE_A`（已访问）和 `PTE_D`（已修改）位，导致硬件认为该页尚未准备好写入而不断抛出异常。通过在 `handle_cow_fault` 中为新 PTE 补齐这两个标志位，彻底解决了重复异常问题。

输出日志不匹配错误：在机制完全正确后，头歌评测依然提示失败，表现为部分预期输出行缺失。通过逐行比对发现，我在重构 `strap.c` 时误删了标准输出要求的 `handle_page_fault: ...` 日志，且在 `do_fork` 中缺少了代码段共享映射的描述日志。解决办法是根据实验预期严格补回这些描述性打印信息，确保在实现优化功能的同时，系统行为的观测记录与标准规范完全一致。

6.3.3 实验收获与建议

通过本次写时复制实验，我深入理解了操作系统高效 `fork` 的实现原理，掌握了 `fork` 时共享物理页、写入时触发页故障再复制的核心思想，明白了页表权限位与自定义 COW 标记位在延迟复制中的关键作用。实验中完成了页表标记、物理页引用计数、COW 页映射与缺页异常处理等模块的编写，串联起进程管理、虚拟内存与异常处理流程。同时，在调试引用计数越界、重复页故障及日志匹配问题的过程中，提升了内核代码调试能力，认识到页表位设置、边界检查与输出规范的重要性，对进程内存共享与隔离机制有了更全面、深刻的认识。

实验七 重载执行

7.1 实验目的

本实验目标是为 PKE 内核引入 `exec` 系统调用，使操作系统具备动态替换进程执行映像的能力。

在之前的实验中，进程的程序内容在创建时即已固定。本实验需要实现重载执行机制：当用户程序调用 `exec` 时，当前进程能够保留其现有的进程标识（如 PID 和 PCB 结构），但通过 VFS 虚拟文件系统从磁盘（HostFS 或 RFS）加载一个新的 ELF 可执行文件，并用其内容彻底替换当前进程的代码段、数据段和堆段。通过这一实验，我们将深入理解进程身份与映像的解耦逻辑，掌握如何通过 VFS 接口在内核态动态解析与加载二进制程序，并学习如何平滑地重置用户态执行上下文，为后续实现功能完备的 Shell 奠定基础。

7.2 实验内容与实验记录

本实验通过实现 `SYS_user_exec` 系统调用，在内核中清理当前进程原有的程序段映射，并利用 VFS 接口重新加载指定路径的 ELF 程序至当前地址空间，最后通过更新 `trapframe->epc` 引导进程跳转至新程序的入口地址执行。

7.3 实验调试及心得

7.3.1 实验原理分析与代码逻辑

`fork` 的本质是复制，而 `exec` 的本质是替换。在操作系统中，进程控制块（PCB）代表了进程的身份，而代码和数据段代表了其映像。实现 `exec` 的思路是：在不改变 PCB 指针和 PID 的前提下，回收当前进程用户态地址空间中与旧程序相关的内存映射，然后按需装载新程序。因此，本文件重点修改 `do_fork()`。主要变化在 `HEAP_SEGMENT` 分支：不再给予进程堆页逐页 `alloc_page + memcpy`，而是让父子进程共享同一物理页并打上 COW 标记。此外，代码段共享映射的日志输出也需要保留。`do_exec()` 的职责可以分成三步：

1. 删除旧程序的用户态映射。
2. 重置堆相关信息。

3. 装入新的 ELF 文件。

具体而言，内核需要首先遍历 `mapped_info` 数组，识别并清理 `CODE_SEGMENT`、`DATA_SEGMENT` 和 `HEAP_SEGMENT`。随后重置堆管理指针（`user_heap_ptr`），确保新程序拥有一个干净的虚拟地址空间起点，从而实现了在同一个进程“外壳”下运行全新的程序逻辑，核心代码如下图所示：

```
int do_exec(const char *pathname) {
    sprintf("User call exec: %s\n", pathname);
    for (int i = 0; i < current->total_mapped_region; i++) {
        switch (current->mapped_info[i].seg_type) {
            case CODE_SEGMENT:
            case DATA_SEGMENT:
                for (int j = 0; j < current->mapped_info[i].npages; j++) {
                    uint64 va = current->mapped_info[i].va + j * PGSIZE;
                    user_vm_unmap((pagetable_t)current->pagetable, va, PGSIZE, 1);
                }
                current->mapped_info[i].va = 0;
                current->mapped_info[i].npages = 0;
                current->mapped_info[i].seg_type = 0;
                break;
            case HEAP_SEGMENT:
                for (uint64 heap_va = current->user_heap.heap_bottom;
                     heap_va < current->user_heap.heap_top;
                     heap_va += PGSIZE) {
                    user_vm_unmap((pagetable_t)current->pagetable, heap_va, PGSIZE, 1);
                }
                current->user_heap.heap_top = current->user_heap.heap_bottom;
                current->user_heap.free_pages_count = 0;
                current->mapped_info[i].npages = 0;
                break;
            default:
                break;
        }
    }
    int write_idx = 0;
    for (int read_idx = 0; read_idx < current->total_mapped_region; read_idx++) {
        if (current->mapped_info[read_idx].va != 0) {
            if (write_idx != read_idx) {
                current->mapped_info[write_idx] = current->mapped_info[read_idx];
            }
            write_idx++;
        }
    }
    current->total_mapped_region = write_idx;
    if (load_bincode_from_fs(current, pathname) != 0) {
        sprintf("Failed to load program: %s\n", pathname);
        return -1;
    }
    return 0;
}
```

图 7-1 do_exec 代码

前面的基础实验中，PKE 启动主程序时通常使用宿主机文件接口，例如直接读取 `./obj/app_exec`。但本次 challenge 的核心目标，是让用户程序能够通过路径名执行 `/bin/app_ls` 这类已经位于文件系统中的程序。这意味着内核不能只依赖宿主机文件接口，还需要能够通过 VFS 接口访问挂载在 `/bin` 下的可执行文件。思路是为 ELF 加载器提供一套“文件系统后端”，利用 `vfs_open` 和 `vfs_read` 代替直接的宿主机文件访问。

据此，在 `kernel/elf.c` 中新增 `load_bincode_from_fs` 函数。它通过 VFS 接口解析新 ELF 文件的程序头，将各个段逐一映射到当前进程的页表中。这种实现方式使 `exec` 能够脱离宿主机环境，真正运行位于模拟器磁盘映像上的程序。

```

int load_bincode_from_fs(process *p, const char *pathname) {
    elf_ctx elfloader;
    elf_vfs_info info;
    info.f = vfs_open(pathname, O_RDONLY);
    if (info.f == NULL) {
        sprint("Fail on opening the file: %s\n", pathname);
        return -1;
    }
    info.p = p;
    if (elf_init_vfs(&elfloader, &info) != EL_OK) {
        sprint("fail to init elfloader.\n");
        vfs_close(info.f);
        return -1;
    }
    if (elf_load_vfs(&elfloader) != EL_OK) {
        sprint("Fail on loading elf.\n");
        vfs_close(info.f);
        return -1;
    }
    p->trapframe->epc = elfloader.ehdr.entry;
    vfs_close(info.f);
    sprint("Application program entry point (virtual address): 0x%lx\n", p->trapframe->epc);
    return 0;
}

```

图 7-2 load_bincode_from_fs 代码

还要记得要处理用户态参数传递与地址翻译问题，exec 的参数（路径名字符串）位于用户态虚拟地址空间中。内核在处理系统调用时，不能直接解引用用户指针，因为内核态和用户态的地址映射不同。必须先将用户虚拟地址翻译为物理地址，才能读取到路径字符串。内核在 sys_user_exec 中调用 user_va_to_pa 函数。通过遍历当前进程的页表找到物理页，将用户态的路径名（如 /bin/app_ls）拷贝到内核缓冲区中。只有这样，内核后续的 VFS 打开操作才能获得正确的路径名，避免因地址无效导致的加载失败。

```

ssize_t sys_user_exec(char *pathname_va) {
    char *pathname = (char*)user_va_to_pa((pagetable_t)
current->pagetable), (void*)pathname_va);

    return do_exec(pathname);
}

```

而其中 user_va_to_pa()函数在上学期的基础实验中已经实现过了。

当新程序装载完毕后，进程不能回到旧程序的 ecall 下一条指令继续执行。必须强行修改进程的返回地址，使其重头开始运行新程序。内核解析新 ELF 文件的 e_entry 字段，并将其直接赋值给 current->trapframe->epc。当 do_exec 完成并触发系统调用返回时，return_to_user 汇编指令会从 trapframe 中恢复 EPC，使 CPU 跳转到新程序的起始位置，从而完成了从旧程序到新程序的平滑切换。

7.3.2 实验调试与问题解决

在实验过程中，我遇到了以下核心问题并进行了针对性调试：

用户态路径名读取失败：在初期实现中，我直接尝试在内核中访问 `pathname` 指针，导致系统因非法地址访问而 `panic`；通过分析发现 `pathname` 是用户虚拟地址，内核无法直接读取，解决方法是引入 `user_va_to_pa` 进行页表查询，将用户态字符串先翻译并拷贝到内核稳定区域，确保了加载路径的正确解析。

主程序加载路径不兼容：本地测试通过但头歌评测报错，提示 `Fail on opening`；原因在于本地使用宿主机路径启动，而线上平台使用 `/bin/app_exec` 这种 VFS 路径启动，原有的 `spike_file_open` 无法处理；通过重构 `load_bincode_from_host_elf` 逻辑，增加对 `/` 开头路径的识别并分流至 `load_bincode_from_fs`，实现了对两种路径模式的全面兼容。

映射信息数组管理混乱：在多次执行 `exec` 后，系统会出现映射冲突；这是因为在 `do_exec` 清理旧段映射后，`mapped_info` 数组中留下了大量空洞，导致新段加载时索引错误；我通过引入“双指针压缩”算法，在清理旧段后实时压缩 `mapped_info` 数组，保持了映射元数据的紧凑性，消除了由于无效条目残留导致的逻辑故障。

系统调用返回值覆盖参数：新程序运行后 `argc` 始终为 0，导致无法读取参数；经排查发现 `handle_syscall` 统一会将 `do_syscall` 的返回值写回 `a0` 寄存器，覆盖了 `do_exec` 已经布置好的参数寄存器；我在 `strap.c` 中对 `SYS_user_exec` 进行了特判，成功时跳过 `a0` 的回写操作，确保了新程序能够正确接收到命令行参数

7.3.3 实验收获与建议

通过本次实验，我在操作系统底层知识和实践能力上都有了显著提升。实验前，我对 `exec` 的理解仅停留在表面概念，经过亲手实现 `exec` 重载执行机制，我清晰区分了 `exec` 与 `fork` 的核心差异：`fork` 是生成新进程，而 `exec` 是替换当前进程的内容，二者底层运行机制截然不同。

同时，我也明确了“进程身份”与“进程映像”的区别，进程控制块、`pid` 等属于进程身份，不会被 `exec` 替换，而代码段、数据段等进程映像则会被新程序替换。在实现过程中，我亲手补齐 VFS 版本 ELF 加载器，完整掌握了 ELF 装载的全流程，也深刻认识到文件系统与程序装载的紧密关联，理解了操作系统各模块的协作逻辑。

此外，实验还让我收获了实用的调试经验，深刻体会到本地测试通过不等于线上通过，学会了按调用链分层排查问题的高效方法，不再盲目修改代码，这也提升了我编写健壮内核代码的能力，对操作系统底层运行机制的认知更加全面深入。

实验八 简易 Shell

8.1 实验目的

本实验要求在前一个挑战实验已经完成 `exec` 的基础上，进一步实现一个简易 Shell,深度整合并验证 PKE 内核中的进程管理、虚拟内存管理以及文件系统三大核心模块的协同工作能力。

通过本实验，我们将深入探究进程执行身份与程序映像的本质区别，掌握在复杂内存布局下维护内核元数据一致性的工程方法，最终实现一个能够自动化解析执行脚本指令的系统环境。

8.2 实验内容与实验记录

本实验通过在内核中完善进程同步状态机（引入 ZOMBIE 状态）与带参数的镜像重载机制，实现了驱动子进程按序加载并执行 `mkdir`、`touch`、`echo` 等外部程序，最终完成对 `shellrc` 自动化脚本的闭环处理

8.3 实验调试及心得

8.3.1 实验原理分析与代码逻辑

从表面上看，Shell 只是读一条命令并执行，但从操作系统视角看，一条命令的完整执行依赖 `fork`、`exec`、`wait` 三大核心机制的协同工作，三者构成了 Unix 风格 Shell 执行的基础模型。

`fork()` 是实现进程创建的核心，父进程调用 `fork` 后，内核会为子进程复制一份完整的执行上下文，包括代码、数据、堆、栈和 `trapframe` 等资源。父子进程从 `fork()` 处同时返回，仅通过返回值区分身份。在代码逻辑中，`do_fork()` 需遍历进程地址空间的各段，逐页完成映射复制，尤其要处理数据段非页对齐、代码段与数据段共享物理页的特殊场景，确保子进程地址空间完整且权限正确。

子进程并不会执行 Shell 程序的后续逻辑，而是通过 `exec(pathname, arg)` 替换自身的程序映像。`exec` 执行时，内核会先清理当前进程的旧地址空间（按页对齐方式释放代码段、数据段等），再通过 VFS 打开指定路径的 ELF 程序，由 ELF 加载器将新程序的段映射到进程页表中，最终修改 `epc` 寄存器跳转到新程序入口。为支持命令参数传递，`exec` 需将参数构造到用户栈上，设置

argc/argv 并保证 trap 返回时不覆盖 a0/a1 寄存器（分别存储 argc 和 argv）。

之前，我们实现的 exec 支持一个参数，但本实验的命令程序几乎都需要一个路径参数，由于 exec 会在加载新程序前清理旧地址空间，如果直接使用用户指针会导致非法访问。因此思路是：先将参数字符串备份到内核，在加载新 ELF 映像并重置堆栈后，手动修改子进程的栈指针（SP），将参数内容压入新程序的栈顶，模拟 C 语言 main(argc, argv) 的压栈过程。

```
// user/user_lib.h
int exec(const char *pathname, const char *arg);
int wait(int pid);
// user/user_lib.c
int exec(const char *pathname, const char *arg) {
    return do_user_call(SYS_user_exec, (uint64)pathname, (uint64)arg, 0, 0, 0, 0, 0);
}
int wait(int pid) {
    return do_user_call(SYS_user_wait, pid, 0, 0, 0, 0, 0, 0);
}
// user/app_shell.c
int pid = fork();
if (pid == 0) {
    int ret = exec(command, para);
    if (ret == -1) printu("exec failed!\n");
} else {
    wait(pid);
}
```

父进程（Shell）为了保证 Shell 脚本中的命令能够按顺序执行，父进程在 fork 出子进程后不能立即读取下一条命令，而必须需通过 wait(pid) 阻塞等待子进程执行完毕，避免多条命令并发执行导致输出错乱或文件系统操作冲突。

为此需要引入 ZOMBIE（僵尸）状态：当子进程调用 exit 时，不应立即释放所有资源，而应保留其 PCB 以供父进程查询状态。父进程通过 wait 系统调

用挂起，直到子进程变为 ZOMBIE 时被唤醒，从而实现执行流的串行化。

在 kernel/process.c 中实现 do_wait(pid)，若目标进程未退出，则将 current 设为 BLOCKED 并执行 schedule()。在 sys_user_exit 中，子进程将状态设为 ZOMBIE 并唤醒父进程，确保了 Shell 能够精确控制命令的生命周期。

```
int do_wait(int pid) {
    process *child = NULL;
    for (int i = 0; i < NPROC; i++) {
        if (procs[i].status != FREE && procs[i].pid == pid) {
            child = &procs[i];
            break;
        }
    }
    if (child == NULL) return -1;
    if (child->parent != current) return -2;
    while (child->status != ZOMBIE) {
        current->status = BLOCKED;
        schedule();
    }
    free_process(child);
    return 0;
}
```

此外，进程地址空间的元数据管理（mapped_info 数组）是所有操作的基础，尤其是 HEAP_SEGMENT 的初始化：需明确堆段的虚拟地址起始位置、页数和类型，否则堆分配时会错误修改代码段页数，进而导致 fork 复制的子进程地址空间污染，引发后续一系列连锁错误。

在 alloc_process 中我显式恢复了 HEAP_SEGMENT 的预留逻辑。这保证了后续 sys_user_allocate_page 增加堆页数时，修改的是 mapped_inf 而非错误的 CODE_SEGMENT 索引，从而维持了地址空间映射表的逻辑一致性，关键代码如下：

```
// initialize the process's heap manager
procs[i].user_heap.heap_top = USER_FREE_ADDRESS_START;
```

```

procs[i].user_heap.heap_bottom = USER_FREE_ADDRESS_START;
procs[i].user_heap.free_pages_count = 0;
// map user heap in userspace
procs[i].mapped_info[HEAP_SEGMENT].va = USER_FREE_ADDRESS_S
TART;

procs[i].mapped_info[HEAP_SEGMENT].npages = 0;
procs[i].mapped_info[HEAP_SEGMENT].seg_type = HEAP_SEGMENT;
procs[i].total_mapped_region = 4;

```

还要注意本实验中的 ELF 数据段可能不是页对齐的，而且代码段和数据段还可能落在同一物理页中。如果 `do_fork()` 像处理代码段那样只做简单共享映射，则会触发映射冲突或导致父子进程意外共享可写数据等问题。因此对 `DATA_SEGMENT` 的复制必须更谨慎，逻辑改为：检查目标页是否已被映射；若已被代码段共享映射，则强行为子进程分配新页并拷贝父进程内容，再赋予 `PTE_W` 权限。这种按需分离的设计确保了非对齐段在物理层面的绝对隔离。

```

uint64 data_start_va = parent->mapped_info[i].va;
uint64 data_page_start = ROUNDOWN(data_start_va, PGSIZE);
uint64 data_end_va = data_start_va + parent->mapped_info[i].npages * PGSIZE;
uint64 actual_npages = (ROUNDUP(data_end_va, PGSIZE) - data_page_start) / PGSIZE;
for (uint64 page_va = data_page_start;
    page_va < data_page_start + actual_npages * PGSIZE;
    page_va += PGSIZE)
{
    pte_t *child_pte = page_walk((pagetable_t)child->pagetable, page_va, 0);
    if (child_pte != NULL && (*child_pte & PTE_V)) {
        uint64 existing_pa = PTE2PA(*child_pte);
        void *new_pa = alloc_page();
        memcpy(new_pa, (void *)existing_pa, PGSIZE);
        user_vm_unmap((pagetable_t)child->pagetable, page_va, PGSIZE, 0);
        user_vm_map((pagetable_t)child->pagetable, page_va, PGSIZE, (uint64)new_pa,
                    prot_to_type(PT_WRITE | PT_READ | PT_EXEC, 1));
        continue;
    }
    uint64 parent_pa = lookup_pa(parent->pagetable, page_va);
    if (parent_pa == 0) continue;
    void *child_pa = alloc_page();
    memcpy(child_pa, (void *)parent_pa, PGSIZE);
    user_vm_map((pagetable_t)child->pagetable, page_va, PGSIZE, (uint64)child_pa,
                prot_to_type(PT_WRITE | PT_READ, 1));
}

```

图 8-1 `do_fork` 关键代码

8.3.2 实验调试与问题解决

在实验过程中，我遇到了以下核心问题并进行了针对性调试：

HEAP_SEGMENT 未初始化引发的级联崩溃：系统在子进程执行第一次 `exec` 进入 `trap` 时触发了 `Misaligned AMO!` 异常，经过 `mepc` 和 `mtval` 的追

踪，发现 `sscratch` 寄存器被污染成了十六进制值 `0x5`。分析发现 `0x5e` 正好是 `SYS_user_exec` 的系统调用号，究其根源是 `alloc_process` 中注释掉了堆初始化代码，导致 `mapped_info` 索引错位，使得堆分配函数误改了代码段的页数记录，最终在 `fork` 时破坏了子进程保存现场所需的元数据，导致在 `smode_trap_vector` 中写入了非法地址。通过恢复 `HEAP_SEGMENT` 的登记逻辑，彻底切断了这条从元数据污染到硬件异常的传播链。

代码段与数据段页重叠导致的映射冲突：在完善 `do_fork` 时频繁遇到 `map_pages fails` 报错，通过打印调试信息发现 `DATA_SEGMENT` 的起始虚拟地址并非页对齐（如 `0x11a08`），且其所在的物理页已被 `CODE_SEGMENT` 提前映射。由于内核不允许对已映射地址再次执行 `user_vm_map`，导致子进程创建失败。解决方法是在 `do_fork` 中引入页边界判定逻辑，检测到重叠页时强制执行“申请新物理页，拷贝内容，重新映射为可写”的一系列动作，从而在逻辑地址重合的情况下实现了父子进程物理数据的完全独立。

exec 参数字符串因地址空间清理导致的 use-after-free：初步实现带参数的 `exec` 时，新程序加载后读取的参数始终是乱码或空字符串。分析调用链后发现，参数指针 `arg` 最初指向父进程的堆空间，而 `do_exec` 的第一步操作就是回收当前进程旧的代码、数据和堆映射。这就导致在解析新 ELF 之前，存储参数的物理页已被释放，形成了一次典型的“释放后使用”错误。通过在进入 `do_exec` 前将参数字符串预先拷贝到内核栈的本地缓冲区，成功解决了地址空间切换过程中的数据保全问题。

系统调用返回值回写覆盖新程序 argc 参数：在 Shell 启动后，虽然程序能切换，但所有命令程序（如 `mkdir`）获取的 `argc` 始终为 0，导致功能失效。排查 `kernel/strap.c` 发现，`handle_syscall` 在系统调用返回前会无差别地执行 `tf->regs.a0 = do_syscall(...)`，将返回值写回 `a0` 寄存器。对于 `exec` 而言，`do_exec` 已经精心布置好了 `a0`（存储 `argc`）和 `a1`（存储 `argv`），这一回写动作覆盖了已布置好的参数。通过在 `strap.c` 中增加针对 `SYS_user_exec` 成功时的特判逻辑，跳过 `a0` 的回写操作，确保了寄存器上下文在跨映像切换时的正确传递。

8.3.3 实验收获与建议

通过本实验，我对 `fork/exec/wait` 三者关系理解更深，过去的基础实验和挑战实验中虽然也有接触三者，而这次真正写出来之后，我更清楚地理解到它们是一套完整命令执行机制。

此外，我对进程地址空间元数据的重要性有了更直观认识，这次最关键的

bug 并不是算法错误，而是 `mapped_info` 中一个段位信息没有初始化。这让我认识到在内核里，很多结构体字段看似只是记录信息，实际上却是后续所有逻辑的基础假设。一旦这个假设失真，一个微小的元数据初始化疏忽，会在经历多次调度和陷阱处理后，在毫不相干的汇编指令处爆发。

建议在以后的实验文档中，可以引导学生先编写一些方便测试，调试的辅助函数，或介绍一些调试命令。比如实现 `print_mapped_info` 辅助函数，能够一键打印当前进程所有段的 `va`、`pa`、`npages` 和权限，这将极大缩短排查元数据污染问题的时间。